

SENTINEL

AI-Driven Anomaly Detection in Component Burn-In & Screening
Team Blueprint, Technical Playbook & Build Plan

PRIMARY DOMAIN: AI / ML

RELIABILITY ENGINEERING

TIME-SERIES

EXPLAINABLE AI

A predictive screening layer that catches latent defects in high-reliability electronic components — parts that pass every datasheet limit but are quietly degrading — before they are soldered into a payload that cannot be repaired.

One-line definition. SENTINEL is an explainable machine-learning screening system that learns each production lot's own normal behaviour, detects components whose parametric drift is anomalous relative to their peers, and forecasts a part's 168-hour degradation from its first 24 hours of burn-in data — so marginal parts are rejected early instead of failing in orbit.

0. How to use this document

Read sections 1–4 as a team on day zero; everyone must be able to explain the physics, not just the code. Sections 5–12 are the technical core — the ML leads own these. Sections 13–19 are the build plan. Section 20 explains how to get maximum leverage out of Claude during the event. Section 22 is the reading list.

Two runnable Python files ship with this document: `generate_burnin_dataset.py` (a physics-based synthetic dataset generator) and `baseline.py` (a working end-to-end baseline for both modules). Run them in the first hour so nobody is blocked on data.

1. Executive Summary

Electronic components destined for space, defence, medical implants and automotive safety systems go through **Environmental Stress Screening (ESS)**. The most important step is **burn-in**: the part is powered and operated at an elevated temperature — typically 125 °C — for 96 to 240 hours, while electrical parameters are read and recorded at intervals such as 0h, 24h, 96h and 168h. The purpose is to force *infant mortality* failures to occur in the lab rather than in the field.

Screening today is judged almost entirely on **static pass/fail limits** taken from the datasheet. A part is good if its standby current stays under, say, 50 µA. The flaw is obvious once stated: if a lot's parts all sit around 10 µA and one part climbs steadily from 12 µA to 32 µA over 168 hours, that part is screaming that something is physically wrong inside it — a partially-open bond, a resistive via, mobile ionic contamination, gate-oxide damage — yet it ships as a good part because 32 < 50. This is a **latent defect**, and it is the dominant cause of early field failures in high-reliability electronics.

SENTINEL replaces the single static threshold with three cooperating layers:

- Module A — Dynamic outlier detection.** Compute each lot's own robust statistical envelope and flag parts that are abnormal *relative to their peers*, not relative to the datasheet.
- Module B — Drift forecasting.** From only `Value_0h` and `Value_24h`, predict `Value_168h`. If the predicted drift rate exceeds a computed safety slope, reject the part at hour 24 — saving 144 hours of oven time per lot.
- Explainability layer.** Every verdict is delivered to the QA inspector as a numbered reason code, a ranked feature contribution, and a plot — not a probability from a black box.

Why the stakes justify aggressive screening. Burn-in is an accelerated-life test governed by the Arrhenius relation. With an activation energy of 0.7 eV, 168 hours at 125 °C is roughly equivalent to **18 years** of operation at 25 °C (acceleration factor ≈ 937). A drift you can see in a week of burn-in is a drift that will consume the part's entire mission life. And in a satellite, there is no maintenance window.

2. Domain Primer — Burn-In in Five Minutes

You do not need to be a semiconductor physicist, but you must not sound like a tourist to the judges. Here is the minimum.

2.1 Where burn-in sits in the screening flow

For military and space-grade microcircuits, the screening sequence is defined by MIL-STD-883 Method 5004 and MIL-PRF-38535. The flow runs through internal visual inspection, temperature cycling, constant acceleration, particle impact noise detection, serialisation, pre-burn-in electrical parameters, burn-in itself, interim post-burn-in electrical parameters, a Percent Defective Allowable computation, final electrical tests, seal tests and external visual inspection. Our project inserts itself at exactly one point: between the recorded electrical parameters and the accept/reject decision.

Element	What it means
MIL-STD-883 Method 1015	The burn-in test method itself. Its stated purpose is to eliminate marginal devices — those with inherent defects or defects from manufacturing aberrations, which show up as time- and stress-dependent failures and would otherwise cause infant-mortality failures in normal use.
Duration and temperature	Class B parts get 160 hours at 125 °C minimum; Class S (space) parts get 240 hours at 125 °C minimum plus a reverse-bias burn-in. The 0/24/96/168h schedule in our problem statement is a standard interim-read cadence.
Read & Record with Deltas	Parameters are not just checked against limits — the <i>change</i> from pre-burn-in to post-burn-in is recorded per part. This "delta" is the closest thing industry has to what we are building, and it is still a fixed threshold.
PDA (Percent Defective Allowable)	A lot-level gate: if more than roughly 5% of parts fail post-burn-in (3% for functional parameters at 25 °C for Class S), the entire lot is rejected. This is why over-rejection is expensive — a trigger-happy model can scrap a good lot.
Outlier identification	High-reliability flows already include an outlier-removal step whose intent is "remove all outliers" — but the industry implementation is a fixed statistical rule. That is the gap SENTINEL fills with learning.

2.2 The parameters you will be modelling

Parameter	What it physically measures	Why it drifts (the defect signature)
Iddq (quiescent supply current)	Current drawn by a CMOS device when it is idle and nothing should be switching. Ideally near-zero leakage only.	The single most sensitive defect indicator in CMOS. Rises with gate-oxide damage, bridging defects, punch-through, ionic contamination. A part whose Iddq climbs during burn-in has a defect that is growing.
Leakage current (I_{leak} , I_{GSS} , I_{DSS})	Current across a junction or insulator that should be blocking.	Indicates dielectric degradation, surface contamination, moisture ingress in the package, hot-carrier damage.
Propagation delay (t_{pd})	Time for a signal edge to traverse the device.	Increases with resistive interconnect, electromigration voiding, partially-open bond wires, threshold-voltage shift (NBTI).
Output levels (V_{OL} , V_{OH})	Voltage the driver can pull the output to.	Degrades with drive-transistor weakening and contact resistance growth.

The one insight that will make your model work: currents are **log-normally** distributed, not normally distributed. Leakage spans orders of magnitude across a population. If you compute a mean and standard deviation on raw microamps, the distribution's long right tail inflates σ , the limits widen, and real outliers hide inside them. **Take log(x) first for every current parameter**, then apply robust statistics. Timing and voltage parameters are close enough to normal to use raw. Say this sentence to a judge and you will sound like you have done this before.

3. Glossary — shared vocabulary for the team

Term	Meaning
ESS	Environmental Stress Screening — the family of stress tests (thermal, mechanical, electrical) used to precipitate weak parts.
Latent defect	A part that passes all absolute limits but carries a physical flaw that will grow. Our target class.
Escape	A defective part that passes screening and ships. A false negative . In space applications this is the catastrophic outcome.
Overkill	A good part rejected by the screen. A false positive . Expensive but recoverable — this is the currency you trade to buy recall.
PAT / DPAT	Part Average Testing / Dynamic PAT. A statistical method that fails parts whose characteristics sit significantly outside the normal distribution of other parts in the same lot, on the reasoning that such a difference — even inside program limits — indicates potential for early failure. Dynamic PAT computes limits per lot in real time as robust mean $\pm$ 6 robust sigma, eliminating lot-to-lot variation.
Robust sigma	A spread estimate that outliers cannot corrupt: $1.4826 \times \text{MAD}$, or $\text{IQR} \div 1.35$. Both equal the true σ for normal data.
MAD	Median Absolute Deviation: $\text{median}(x - \text{median}(x))$.
Delta	The change in a parameter between two read points, e.g. $\text{Value_168h} - \text{Value_0h}$.
Drift rate / slope	Delta divided by elapsed hours. Our Module B forecasts it.
Safety slope	The maximum drift rate a part may show and still be trusted for the mission. Defined in §8.4.
USL / LSL	Upper / Lower Specification Limit — the datasheet numbers.
Masking / swamping	Two classic failure modes of mean-based outlier detection: a big outlier inflates σ so it hides itself (masking), or drags the mean so that good parts look bad (swamping). Robust statistics exist to prevent both.

4. The Problem Statement, Decoded

Strip the prose away and the organisers are asking for four deliverables. Build to this list.

#	Asked for	What that actually means you must ship
1	Module A: dynamic outlier detection	A function that, given a lot of parts, returns a per-part anomaly score that is computed <i>from the lot itself</i> . The worked example in the brief — mean 10 μA, one part at 45 μA, datasheet max 50 μA — must be caught. Your demo must show this exact case being caught.
2	Module B: time-series drift predictor	A regression model with inputs restricted to Value_0h and Value_24h that outputs a predicted Value_168h, plus a rule that converts predicted slope into an early-reject decision.
3	Anomaly detection score, false-negative weighted	They will score you with an asymmetric metric. Do not optimise accuracy or F1. Optimise recall subject to an overkill budget, and report F _β with β ≥ 2. Tune your decision threshold explicitly for this, and say <i>so on the slide</i> .
4	Explainability	A QA inspector must be able to read the verdict. Ship a per-part report with numbered reason codes, ranked contributions, and a drift plot. This is a full third of the marking scheme and it is the cheapest of the three to win.

The trap. Most teams will train an autoencoder or an Isolation Forest on raw values, report a 0.94 accuracy, and lose. Accuracy is meaningless when defects are 5–8% of the population — a model that flags nothing scores 93%. And a black-box score fails the third criterion outright. Your advantage is **a glass-box scoring architecture with an explicitly asymmetric decision rule**.

5. Solution Architecture

```
Parametric CSV (0h / 24h / 96h / 168h) | v [1] INGEST & VALIDATE ---- schema check, unit check, duplicate serials, missing reads | v [2]
TRANSFORM ----- log() for currents, per-lot normalisation, delta + slope features | +-----+ | | v v [3] MODULE
A [4] MODULE B Dynamic outlier detection Drift forecast from 0h + 24h L1 static limits physics-informed power law L2 robust univariate (DPAT) +
gradient-boosted residual L3 multivariate / correlation + quantile upper bound L4 learned ensemble -> predicted Value_168h | -> predicted slope
vs safety slope +-----+ | v [5] FUSION -> Screening Risk Score 0-100 verdict: ACCEPT / WATCH / REJECT | v [6]
EXPLAINABILITY ENGINE reason codes, SHAP contributions, drift plot, nearest-healthy-peer comparison, counterfactual | v [7] QA INSPECTOR
DASHBOARD + signed screening report (PDF)
```

5.1 Design principles (put these on a slide)

- **Glass box by construction.** The final score is a weighted sum of named sub-scores. ML improves the sub-scores; it never becomes the top-level decision.
- **The lot is the reference, not the datasheet.** Every statistic is computed within a lot/wafer group.
- **Asymmetric by design.** Thresholds are chosen on a cost matrix where a miss costs 100× an over-reject, and that ratio is a visible slider in the UI.
- **Physics before parameters.** A power-law degradation model with one fitted exponent beats a 200-parameter network when you only have two input time points.
- **Every rejection is auditable.** The system emits a record a QA engineer could defend in a design review.

6. The Data Problem — and how to solve it in an hour

You will almost certainly not be handed a real burn-in dataset with labelled latent defects; that data is proprietary and, for space parts, often export-controlled. Two possibilities:

- **Organisers provide a CSV.** Then your job is to profile it hard in the first hour: distributions per parameter, per lot, log vs linear, missingness, label balance.
- **They don't.** Then you generate your own — *and label it clearly as simulated*. A well-argued synthetic dataset is a strength, not an apology: it lets you prove recall against known ground truth, which real unlabelled data can never do.

6.1 The generator that ships with this document

```
generate_burnin_dataset.py produces 2,100 parts across 6 lots, each with 4 parameters read at 0h / 24h / 96h / 168h. Physics encoded:

X(t) = X0 · ( 1 + A · (t / 168)n ) healthy parts A ≈ 0.03 with a heavy right tail n ≈ 0.35–0.75 (sub-linear: settling) latent defects A ≈ 0.18–
1.10 n ≈ 0.80–1.60 (super-linear: accelerating) gross failures A ≈ 2.5–6.0 n ≈ 1.0–2.0 (breach the datasheet by 168h)
```

Deliberate realism that makes the problem hard and your solution meaningful:

- **Currents are lognormal, timing is normal** — so naive σ-based limits genuinely fail.
- **Lot-to-lot offsets**, and one deliberately bad lot (L04) with shifted centre, wider spread and 3× the defect rate — this is what proves dynamic limits beat static ones.
- **Overlapping tails.** Roughly 10% of healthy parts are naturally "wide" and will look suspicious. Without this overlap the task is trivial and your metrics are worthless.
- **Localised defect signatures.** Each latent defect affects only one or two of the four parameters — so a detector that only watches Iddq misses half the population, and multivariate methods earn their place.
- **1.5% measurement noise** and a small number of dropped 96h reads.
- **A held-out split** mimicking the organisers' hidden ground truth: `public_train.csv` keeps the labels and 168h values, `hidden_test.csv` strips them, `ground_truth.csv` holds them for scoring.

Headline statistic from the generated data — use this in the pitch. Of the 174 latent defects, **100% pass every static datasheet limit at 168 hours**. Static screening catches none of them. That single number is your entire problem statement, quantified.

6.2 If the organisers give you real data instead

Keep the generator anyway. Use it for unit tests, for the demo (you control the story), and for stress-testing the model against defect rates and lot shifts you did not observe. Then retrain on the real data and report both.

7. Module A — Dynamic Outlier Detection

Build this in four layers, each of which independently works and independently demos. Ship L1 and L2 by hour 8; L3 and L4 are the differentiators.

L1 — Static limits (the baseline you exist to beat)

Flag if any value exceeds the datasheet USL/LSL at any read point. Keep it in the pipeline forever — not because it is good, but because your demo needs to show it catching almost nothing.

L2 — Dynamic PAT: robust univariate limits per lot

This is the industry-standard method the brief is gesturing at, and it should be your first real model. Compute, per lot and per parameter and per read point:

```
med = median(x)
MAD = median(|x - med|)
sigma* = 1.4826 * MAD # 1.4826 makes MAD a sigma-equivalent under normality
# alternative: sigma* = IQR / 1.35
robust_z(xi) = (xi - med) / sigma*
DPAT limits = med ± 6 * sigma*
# AEC-Q001 uses 6 robust sigma flag if |robust_z| > k
# k tuned on the FN-weighted metric, typically 4.5–6
```

Apply it not only to the raw value but to **four derived views** per parameter — this multiplies your detection surface at zero cost:

View	Feature	Catches
Absolute level	$z(\log V_{0h}), z(\log V_{168h})$	The brief's worked example: a 45 μ A part in a 10 μ A lot.
Early delta	$z(V_{24h} - V_{0h})$	Parts that start normal but move fast. The signal Module B is built on.
Total drift	$z(V_{168h} - V_{0h})$	The classic delta criterion, made lot-relative.
Curvature	$z \text{ of } (V_{168h} - V_{96h}) / (V_{24h} - V_{0h})$	Accelerating degradation — healthy parts settle, defects accelerate. This ratio is the cleanest physical separator you have.

Do this on log-scale for currents. Compute the robust z on $\log(x)$, not x . A part at 45 μ A in a lot centred at 10 μ A with log- σ 0.18 sits at robust-z $\approx$ 8.4 on log scale — unmissable — while on the raw scale the lognormal tail may have inflated σ enough to hide it.

L3 — Multivariate: catching the correlation break

A part can be within limits on every individual parameter and still be obviously wrong as a combination — Iddq up 40% while propagation delay is unchanged is not a pattern any healthy part produces. Three techniques, in increasing order of ambition:

Method	How to use it here	Notes
Robust Mahalanobis distance (with Minimum Covariance Determinant)	Fit <code>sklearn.covariance.MinCovDet</code> per lot on the delta-vector [Δ Iddq, Δ Ileak, Δ tpd, Δ Vol]. Distance follows χ^2 with 4 dof — threshold at $\chi^2(0.999)$.	The best explainability-to-power ratio: you can decompose the distance per parameter and say exactly which axis drove it.
Isolation Forest	Unsupervised, on the standardised feature matrix. Fast, no distributional assumptions, handles mixed features.	Score is a path length; pair it with SHAP or per-feature isolation depth to keep it explainable.
Local Outlier Factor / autoencoder	LOF catches local density anomalies (a small cluster of subtly odd parts). A tiny autoencoder trained only on healthy parts flags high reconstruction error.	Use only if time permits; the autoencoder is the hardest to explain and the least likely to beat robust stats on tabular data of this size.

Optional, high-impact if you have wafer coordinates: implement **nearest-neighbour residual detection** ("Good Die in a Bad Neighbourhood"). Compare each die's value to the median of its 8 physical neighbours on the wafer map. Spatial clustering of defects is real, and a wafer-map heat overlay in the dashboard is the single most photogenic thing you can put on screen.

L4 — Supervised ensemble (only if you have labels)

If your dataset has ground-truth labels, train a gradient-boosted classifier (XGBoost or LightGBM) on the full engineered feature set, with `scale_pos_weight` set to the inverse class ratio so misses are penalised hard. Then **stack it**: the final risk score is a weighted blend of the L2 robust-z score, the L3 distance, and the L4 probability. Report the blend weights on your slide — they are interpretable, and showing them is itself an explainability win.

7.1 Reference implementation of the core

```

import numpy as np, pandas as pd

def robust_z(s: pd.Series) -> pd.Series:
    """Outlier-immune z-score. 1.4826 * MAD == sigma for normal data."""
    med = s.median()
    mad = (s - med).abs().median()
    sigma = 1.4826 * mad if mad > 0 else s.std(ddof=1)
    return (s - med) / (sigma if sigma > 0 else 1.0)

def dpat_limits(s: pd.Series, k: float = 6.0):
    """AEC-Q001 style dynamic limits: robust mean +/- k robust sigma."""
    med = s.median()
    sigma = 1.4826 * (s - med).abs().median()
    return med - k * sigma, med + k * sigma

# per-lot, per-parameter, on log scale for currents
feat = pd.DataFrame(index=df.index)
for p, is_current in PARAMS.items():
    v0, v24, v168 = (np.log(df[f"{p}_{t}h"]) if is_current else df[f"{p}_{t}h"])
    for t in (0, 24, 168):
        for lot, g in df.groupby("lot"):
            i = g.index
            feat.loc[i, f"z_{p}_level"] = robust_z(v0.loc[i])
            feat.loc[i, f"z_{p}_early"] = robust_z((v24 - v0).loc[i])
            feat.loc[i, f"z_{p}_drift"] = robust_z((v168 - v0).loc[i])

anomaly_score = feat.abs().max(axis=1) # worst-case across all views

```

8. Module B — Time-Series Drift Predictor

The constraint is severe and it is the point: **two input points, one output 144 hours later**. With two points you cannot fit a two-parameter curve per part. The trick is to fit the *shape* globally from training data and the *magnitude* per part.

8.1 The physics-informed baseline (build this first — it takes 20 minutes)

Model: $X(t) = X_0 \cdot (1 + A \cdot (t/168)^n)$ Step 1 Fit ONE global exponent n^* per parameter on the training set (grid search n in $[0.3, 2.0]$, minimise MAE against known 168h values). Step 2 Per part, solve for A from the only delta you have: $A_i = (X_{24} / X_0 - 1) / (24/168)^{n^*}$ Step 3 Forecast: $\hat{X}_{168} = X_0 \cdot (1 + A_i)$

This is defensible in a design review, needs no training infrastructure, and on the shipped dataset it beats naive linear extrapolation by roughly 25–65% MAE depending on the parameter. It also gives you a physical quantity — the exponent n — that you can plot and talk about: *healthy parts settle ($n < 1$), defective parts accelerate ($n > 1$)*.

8.2 The learned model (build this second)

Feed a gradient-boosted regressor (or ridge, as a sanity check) a rich feature vector and let it learn the residual the physics model leaves behind:

Feature	Why it carries signal
V_0h, V_24h, log of both	Absolute level; high starting leakage predicts high absolute drift.
V_24h – V_0h and V_24h / V_0h	Absolute and relative early movement. The ratio generalises across lots; the difference carries units.
Physics forecast $\hat{X}_{168}$ from §8.1	Stacking: give the ML model the physics answer as an input and let it correct it. Almost always beats either alone.
Robust z of each of the above, within lot	Converts "is this part high" into "is this part high <i>for this lot</i> " — the lot-relative framing that unifies Modules A and B.
Lot-level aggregates: lot median V_0h, lot median early delta, lot MAD	Lets the model condition on lot health. A part drifting at the lot average is fine; the same drift in a tight lot is not.
Cross-parameter features: $\Delta I_{ddq} / \Delta t_{pd}$	Correlation-break signal, shared with Module A's L3.

8.3 Predict a bound, not a point — this is the false-negative win

Train a **quantile regressor** (LightGBM with `objective="quantile"`, `alpha=0.90`, or scikit-learn's `GradientBoostingRegressor(loss="quantile")`) alongside the mean model. Report the point estimate for the MAE metric, but make the **reject decision on the 90th-percentile upper bound**. A part is rejected if even its optimistic-case forecast breaches the safety slope. This converts your regressor into a conservative screen without hurting your MAE score, and it is a sophisticated answer to "how did you handle the false-negative asymmetry?"

8.4 Defining the safety slope

The brief says to flag parts whose predicted drift rate exceeds "a calculated safety slope" but does not define it. Define it yourself, in two independent ways, and take the stricter:

(a) MISSION-BASED — will the part survive the mission? $S_{mission} = (USL - V_0) / T_{equivalent_hours}$ where $T_{equivalent} = mission_hours / AF$, $AF = \exp(-Ea/k) \cdot (1/T_{use} - 1/T_{stress})$ Arrhenius $Ea \approx 0.7$ eV, $k = 8.617e-5$ eV/K, T in kelvin Worked: $Ea=0.7$, $T_{use}=25$ °C, $T_{stress}=125$ °C -> $AF \approx 937$ so 168 burn-in hours $\approx$ 18 years of field life at 25 °C (b) POPULATION-BASED — is the part normal for its lot? $S_{population} = median(slope_lot) + 4.5 \cdot 1.4826 \cdot MAD(slope_lot)$ SAFETY SLOPE = $\min(S_{mission}, S_{population})$

Presenting both, and explaining that you take the minimum, is a strong slide. (a) says *this part will not survive*; (b) says *this part is not like its siblings*. Either is grounds for rejection.

8.5 The commercial argument for Module B

Because Module B only needs 0h and 24h, a lot can be triaged at hour 24. Burn-in ovens are a scarce, power-hungry, capacity-limiting resource in every hi-rel line. **Removing predicted-bad parts at hour 24 instead of hour 168 frees 144 oven-hours per rejected part and shortens the screening cycle**. Put a cost number on this in the pitch — judges reward teams that connect the model to money, not just to metrics.

9. Fusion — the Screening Risk Score

Collapse everything into one transparent 0-100 number, computed as a weighted sum of named sub-scores. Weights are visible, tunable in the UI, and defensible. This mirrors how reliability engineers already think, and it makes explainability nearly free.

Screening Risk Score = 30% · static_margin_score (how close to the datasheet limit at 168h) + 25% · dynamic_outlier_score (max robust |z| across parameters and views, capped) + 20% · predicted_drift_score (predicted 168h slope / safety slope, capped) + 15% · multivariate_score (robust Mahalanobis distance percentile) + 10% · curvature_score (acceleration ratio: late drift / early drift) Verdict bands (tunable, and the tuning is the point): 0–39 ACCEPT ship 40–69 WATCH ship with serial flagged for extra scrutiny / lot review 70–100 REJECT remove from lot before final electrical test

The three-band verdict is better than binary: it matches how QA actually operates, it softens the overkill cost, and it gives you something to show on a dashboard that a binary flag cannot.

10. Explainability — one third of the marks, and the cheapest to win

The evaluation criterion is literally "*Can the model justify its classification to a QA inspector?*" So build for that persona. Four artefacts:

10.1 Numbered reason codes

Machine-readable, human-obvious, and auditable. Emit them from rules, not from a model:

Code	Trigger	Inspector-facing text
R-101	robust_z(level) > 6 on any parameter	"Iddq at 168h is 8.4 robust sigma above the lot median (31.2 µA vs lot median 10.4 µA). Within datasheet limit of 50 µA but abnormal for this lot."
R-201	robust_z(early delta) > 5	"Leakage rose 18% in the first 24 hours; 95% of this lot rose under 3%."
R-301	predicted slope > safety slope	"Forecast 168h value of 34.1 µA implies a drift rate 2.3× the mission safety slope."
R-401	Mahalanobis > $\chi^2(0.999)$	"Parameter combination is inconsistent: Iddq rising while propagation delay is flat — no healthy part in this lot shows this pattern."
R-501	curvature ratio > 2	"Degradation is accelerating rather than settling (late-window drift is 3.1× the early window)."
R-601	lot-level: flagged fraction > PDA	"Lot L04 has 18% flagged parts, exceeding the 5% PDA threshold — recommend lot-level review."

10.2 Feature attribution

Use `shap.TreeExplainer` on the gradient-boosted models and render a per-part waterfall plot. For the robust-z layers, attribution is free: the contribution *is* the z-score. Show a horizontal bar chart of the top five contributing features with their values and their lot medians side by side.

10.3 The visual that sells it

For any flagged part, plot its four read points against a shaded band showing the lot's 5th–95th percentile envelope over time, with the Module B forecast extended as a dashed line to 168h and the datasheet USL as a red horizontal line. One glance and the story is obvious: the part is walking out of the herd. This single chart will do more work in your demo than any metric.

10.4 The screening report

Generate a one-page PDF per rejected part: serial number, lot, verdict, risk score, reason codes, the plot, measured values against lot statistics, model version and timestamp. Traceability is a hard requirement in real hi-rel QA, and producing a signed artefact makes your project look like a product rather than a notebook.

11. Evaluation Methodology

11.1 Metrics to report (and metrics to refuse)

Metric	Module	Why / how
Recall (sensitivity)	A	Fraction of true latent defects caught. This is the escape metric. Headline it.
F_{β} , $\beta = 2$ or 3	A	Weights recall β^2 times more than precision. State your β and justify it from the cost of a spacecraft failure.
Recall @ fixed overkill	A	The most honest single number: "at a 5% over-rejection budget we catch X% of latent defects." Plot recall vs overkill as a curve.
PR-AUC / Average Precision	A	Threshold-free measure of ranking quality. Correct for imbalanced data; ROC-AUC is over-optimistic here.
Expected cost	A	$C = 100 \cdot \text{FN} + 1 \cdot \text{FP}$, or your own ratio. Choose the threshold that minimises it and show the cost curve.
MAE on Value_168h	B	The organisers' stated metric. Report it per parameter and normalised (MAE / lot median) so it is comparable across parameters.
Tail MAE	B	MAE restricted to the top decile of true drifters. Nobody cares about accuracy on flat parts. Reporting this shows judgement.
Slope-decision accuracy	B	Does the predicted slope put the part on the correct side of the safety slope? This is what the model is actually for.
Plain accuracy	—	Do not report it. With ~8% defects, predicting "all good" scores 92%. If asked, explain why — it is a free credibility point.

11.2 Validation discipline

- **Split by lot, not by part.** A random part-level split leaks lot statistics between train and test and will inflate your scores. Use `GroupKFold(groups=lot)`. Mention this explicitly — judges with ML experience look for exactly this mistake.
- **Never fit scalars or lot statistics on the test set.** Compute DPAT limits within each lot at inference time; that is legitimate (it is unsupervised and uses only the lot in front of you), but any supervised fitting must respect the split.
- **Keep a frozen holdout** you touch exactly once, at the end.
- **Report a confusion matrix** with real counts, not just percentages.

11.3 Threshold selection

```
for thr in candidate_thresholds: cost[thr] = C_FN * false_negatives(thr) + C_FP * false_positives(thr) chosen = argmin(cost) # with C_FN / C_FP = 100 by default Expose C_FN / C_FP as a slider in the dashboard. Moving it live in front of judges, and watching the flagged set grow while recall climbs to 100%, is the most persuasive 15 seconds of your demo.
```

12. Baseline results you must beat

Measured on the shipped synthetic dataset (2,100 parts, 6 lots, 174 latent defects, 63 gross failures). These are real numbers from `baseline.py`, not aspirations.

Method	Recall on latent defects	Precision	F_2	Parts flagged
Static datasheet limits only	0.000	—	0.000	63 (gross only)
Dynamic PAT, robust $ z > 8$	0.414	0.533	0.433	135 (6.4%)
Dynamic PAT, robust $ z > 6$	0.523	0.555	0.529	164 (7.8%)
Dynamic PAT, robust $ z > 4.5$	0.638	0.514	0.609	216 (10.3%)

Ranking quality of the baseline anomaly score: PR-AUC = 0.389. Module B baseline (power-law extrapolation from 0h and 24h only) reduces MAE against linear extrapolation by 27% on Iddq, 26% on leakage and 65% on propagation delay — but is beaten by a naive "last value carried forward" on the two low-drift parameters, which is itself a finding worth reporting.

Your target: recall above 0.85 at an overkill rate under 10%, with every flag explained. Getting there needs L3 multivariate detection and the curvature feature — univariate robust z alone plateaus around 0.65 because roughly a third of the latent defects are subtle in every single dimension and only visible as a combination.

13. System Architecture & Technology Stack

Layer	Build	Recommended tools
Data ingest	CSV/Parquet upload, schema validation, unit checks, duplicate-serial detection	pandas, pydantic or pandera for schema contracts
Feature store	Deterministic feature builder — the same function used in training and inference	Plain Python module, unit-tested. Do not duplicate feature logic in the notebook and the API.
Module A	Robust statistics, MinCovDet, IsolationForest, XGBoost/LightGBM classifier	numpy, scipy, scikit-learn, xgboost or lightgbm
Module B	Power-law fit + gradient-boosted regressor + quantile regressor	scipy.optimize, lightgbm, statsmodels
Explainability	SHAP values, reason-code engine, per-part plots, PDF report	shap, matplotlib/plotly, reportlab or WeasyPrint
Backend API	POST /screen (batch CSV in, verdicts out), GET /part/{serial}, GET /lot/{id}	FastAPI + uvicorn — fast to write, auto-generates OpenAPI docs that impress judges
Storage	Parts, readings, verdicts, model runs, audit log	SQLite for the hackathon; Postgres or Supabase if you want a hosted demo
Dashboard	Lot overview, part drill-down, threshold slider, wafer map	Streamlit if you are short on people (2 hours to a good UI); React + Recharts/Plotly if you have a dedicated frontend developer
Reproducibility	Fixed seeds, requirements.txt, one-command demo script, model version in every verdict	Make a run_demo.sh. Judges will ask you to re-run it.

Streamlit vs React. Unless you have someone whose only job is frontend, use Streamlit. A polished Streamlit app with good charts beats a half-finished React dashboard every single time, and it frees an entire person to work on the models. Spend the saved time on the explainability plots.

14. What to Actually Build — priority order

Do not attempt a production MES integration. Build a complete, polished loop over one simulated production run, and make sure a judge can see the whole thing work end to end.

#	Component	Minimum demo requirement
1	Dataset + EDA	2,000+ parts across 6 lots with known ground truth; distribution plots showing lognormality and lot shift
2	Static-limit baseline	Proves static screening catches 0% of latent defects — your problem statement, quantified
3	Module A: DPAT robust z	Catches the brief's worked example (45 µA in a 10 µA lot) and reports recall/precision
4	Module B: drift forecast	Predicts Value_168h from 0h + 24h; MAE reported; scatter of predicted vs actual
5	Safety-slope rule	Converts the forecast into an early-reject decision at hour 24
6	Fusion risk score	0-100 score with ACCEPT / WATCH / REJECT bands
7	Explainability engine	Reason codes + SHAP bar chart + the drift-vs-lot-envelope plot
8	Dashboard	Lot table, sortable by risk; part drill-down; cost-ratio slider
9	Evaluation panel	Confusion matrix, recall-vs-overkill curve, MAE, PR-AUC — live in the app
10	Module A L3 multivariate	Robust Mahalanobis / Isolation Forest lifting recall past the univariate plateau
11	Screening report PDF	One-page auditable artefact per rejected part
12	Wafer-map view (optional)	Spatial defect clustering heat map — high visual impact if you have coordinates

15. Dashboard Screens

Screen	Key elements
Lot Overview	Lot cards with part count, flagged %, PDA status (green/amber/red), mean risk score; alert when a lot exceeds PDA
Part Table	Sortable/filterable by risk score, verdict, lot, parameter; colour-coded verdict chips; click to drill down
Part Detail	The drift plot with lot envelope and forecast; risk score breakdown bar; reason codes; measured values vs lot median table; "download screening report"
Distribution Explorer	Per-parameter histogram on log and linear axes with DPAT limits and datasheet limits overlaid; the flagged part highlighted
Model Performance	Confusion matrix, recall-vs-overkill curve, PR curve, predicted-vs-actual scatter for Module B, MAE table
Decision Policy	The cost-ratio slider (C_FN : C_FP), live threshold, live recall/overkill readout. Your demo centrepiece.
Wafer Map (optional)	Die grid coloured by risk score, showing spatial clustering

16. Data Model

Entity	Fields
Part	serial (PK), lot_id, wafer_id, x, y, device_type, date_code
Reading	id, serial (FK), hours, parameter, value, unit, temperature_C, tester_id, timestamp, valid_flag
Lot	lot_id, part_count, burn_in_condition, start_time, end_time, pda_threshold, pda_actual, status
ParameterSpec	parameter, unit, LSL, USL, delta_limit, distribution_type (normal/lognormal), higher_is_worse
Verdict	id, serial, model_version, risk_score, verdict (ACCEPT/WATCH/REJECT), module_a_score, module_b_score, predicted_168h per parameter, created_at
ReasonCode	id, verdict_id, code, severity, message, contributing_feature, feature_value, lot_reference_value
ModelRun	id, version, trained_at, dataset_hash, hyperparameters, cv_recall, cv_precision, mae_by_parameter

17. Demo Scenario — the eight-minute storyboard

- The stake (30s).** A satellite power controller fails 14 months into a 7-year mission. Root cause: a part with a resistive via that passed every screening test. Cost: the mission. No repair possible.
- The data (45s).** Show lot L04's Iddq distribution. Every single part is under the 50 μ A datasheet limit. Static screening says: 100% pass, ship them all.
- The reveal (45s).** Overlay the dynamic PAT limits computed from the lot itself. A cluster of parts sits far outside the lot's own behaviour while comfortably inside the datasheet. Show the number: **100% of latent defects in this dataset pass static limits.**
- Module A (90s).** Run the detector live. Show the flagged list, then drill into one part: the drift plot with the lot envelope, the part walking out of the herd, the reason codes in plain English.
- Module B (90s).** Now hide everything after hour 24. Run the forecaster. Show predicted vs actual 168h scatter, the MAE, and then the punchline: "we made this call 144 hours early — that is 144 oven-hours per part we did not waste."
- The asymmetry (60s).** Drag the cost slider from 1:1 to 100:1. Watch the threshold move, the flagged set widen, recall climb toward 1.0. Say: "a false negative is a dead satellite; a false positive is a \$40 part. The policy is a business decision, and we made it visible instead of hard-coding it."
- Explainability (60s).** Open the generated screening report PDF for a rejected part. "This is what the QA inspector signs. Serial, verdict, evidence, model version, timestamp."
- Metrics and close (60s).** Confusion matrix, recall-at-overkill curve, MAE table. Close on: "Static limits ask *is this part legal?* SENTINEL asks *is this part behaving like its siblings, and where will it be in a week?*"

18. Team Division & 36-Hour Timeline

Role	Owns
ML Lead — Module A	Robust statistics, DPAT, multivariate detection, ensemble, threshold policy, recall/overkill curves
ML Lead — Module B	Power-law fit, feature engineering, gradient-boosted + quantile regressors, safety slope, MAE evaluation
Data & Backend	Dataset generator, schema validation, feature builder module, FastAPI service, database, unit tests
Frontend / Visualisation	Dashboard, drift plots, wafer map, cost slider, PDF report generator
Domain & Pitch	Reliability research, reason-code wording, slide deck, demo script, timing rehearsal, the cost narrative

Hours	Milestone
0–2	Everyone reads §1–4. Repo created, requirements.txt pinned, dataset generated, EDA notebook running. Agree the feature-name convention <i>now</i> — half of all hackathon merge conflicts are column names.
2–6	Static baseline + DPAT robust z working end to end in a script. First recall number on the board. Frontend skeleton with hard-coded data.
6–12	Module B power-law baseline + first MAE. Module A multivariate layer. FastAPI wrapping both. Checkpoint: the whole pipeline runs on one command.
12–18	Gradient-boosted models, GroupKFold validation, threshold optimisation. Dashboard connected to real outputs.
18–24	Explainability: SHAP, reason codes, the drift plot. This is where marks are won — do not skip it for another 2% recall.
24–28	Fusion score, three-band verdict, cost slider, PDF report. Feature freeze at hour 28.
28–32	Evaluation panel, final numbers, README, architecture diagram. Nothing new is added after this point.
32–36	Slides, demo rehearsal (at least three full run-throughs, timed), backup screen-recording of the demo in case the laptop misbehaves, Q&A prep.

Record a backup video of your working demo at hour 30. Live demos fail. A team that says "the wifi dropped, here's the recording" keeps its score; a team that stands there restarting a server loses it.

19. Pitfalls & Solutions

Pitfall	Fix
Reporting accuracy on imbalanced data	Report recall, $F_{\beta \geq 2}$, PR-AUC and expected cost. Refuse accuracy out loud.
Mean/ σ limits on lognormal currents	Log-transform first, then median and MAD. This alone may be your biggest single recall gain.
Random train/test split leaking lot statistics	<code>GroupKFold(groups=lot)</code> . Say so in the slides.
Black-box model, no justification	Glass-box weighted score on top; ML only inside the sub-scores. Reason codes from rules.
Trying to fit a per-part curve from two points	Fit the exponent globally, the amplitude per part (§8.1).
Over-rejecting and blowing the PDA	Three-band verdict, cost-based threshold, and an explicit overkill budget in every reported result.
Feature logic duplicated between notebook and API, silently diverging	One <code>features.py</code> imported by both. Test it.
Deep learning that underperforms a median	On 2,000 tabular rows, gradient boosting and robust statistics win. Use a neural net only if it beats the baseline on the holdout — and keep the baseline in the deck either way.
Beautiful models, no demo	Freeze features at hour 28. Rehearse three times.
Nobody on the team can answer "what is Iddq?"	\$2 and \$3 exist for this. Judges ask the quiet team member.

20. Getting Maximum Leverage Out of Claude

Treat Claude as a fifth team member with perfect recall and no ego about rewriting its own code. The setup below takes 20 minutes and pays for itself in the first two hours.

20.1 Set-up checklist, in priority order

Do this	Time	Why it matters here
1. Create a Project in Claude and put this document, the problem statement and your dataset schema into its knowledge	5 min	Every chat then starts with full context. No more re-explaining what Iddq is. Put your conventions in the project instructions: column naming, the fact that currents are log-transformed, the metric you optimise.
2. Install Claude Code — <code>npm install -g @anthropic-ai/claude-code</code> , then run <code>claude</code> in the repo (Node 18+)	10 min	The single biggest lever. It reads and edits your actual repo, runs tests, fixes its own errors, and handles git. Docs: docs.claude.com/en/docs/claude-code/overview
3. Write a <code>CLAUDE.md</code> at the repo root	5 min	Claude Code reads it automatically every session. Put in: project goal, directory layout, the metric that matters, "never use mean/std on currents", how to run tests, and the feature-naming convention. This is how you stop it drifting from your team's decisions.
4. Turn on the analysis / code-execution tool in chat settings	1 min	Lets Claude actually run your CSV, compute distributions, and hand back real numbers and charts rather than plausible-looking guesses.
5. Connect GitHub	2 min	Claude can read the repo, review diffs and open PRs. Useful for the teammate who is reviewing rather than writing.
6. Connect Supabase if you host the database	3 min	Claude can inspect the schema and write queries against the real thing instead of an imagined table.
7. Use Artifacts for the dashboard prototype	—	Ask for a working React dashboard with Recharts in a single artifact; iterate on it live, then port to Streamlit or keep it.
8. Claude for Excel if the organisers hand you a spreadsheet	2 min	Profiling, cleaning and pivoting the parametric data without leaving the sheet.
9. Claude Design / PowerPoint for the deck and architecture diagrams	—	Do this at hour 32, not hour 2. Feed it your final numbers.
10. Extended thinking for algorithm design decisions	—	Use it for "which detector should own which failure mode", threshold policy design, and debugging why recall is stuck. Not needed for boilerplate.

20.2 A prompting playbook for this specific project

Order matters. Ask for the evaluation harness *before* the models — otherwise you will build models you cannot compare.

1. "Here is my dataset schema and 20 sample rows. Profile it: distribution shape per parameter, lognormal vs normal, lot-to-lot shift, missingness, class balance. Then tell me which three features will carry the most signal and why."
2. "Write evaluate.py: given y_true and a continuous anomaly score, produce recall, precision, F2, PR-AUC, the recall-vs-overkill curve, and the cost-minimising threshold for C_FN/C_FP = 100. Group-aware by lot. Write tests."
3. "Implement Module A layers L1 and L2 exactly as specified in my project doc. Log-transform currents. Robust z per lot. Return a tidy DataFrame with one column per view so I can inspect contributions."
4. "My recall is stuck at 0.64 with univariate robust z. Here is the confusion matrix and 10 missed parts with all their features. Diagnose why they are being missed and propose the smallest change that fixes it." <-- the highest-value prompt you will write
5. "Add a robust Mahalanobis layer on the delta vector using MinCovDet, per lot. Decompose the distance per parameter so I can show which axis drove each flag."
6. "Build the reason-code engine: given a part's feature row and the lot statistics, emit codes R-101 through R-601 with inspector-facing English including the actual numbers and the lot reference values."
7. "Generate the per-part drift plot: four read points, lot 5th-95th percentile band, dashed forecast to 168h, red USL line, annotated risk score."
8. "Review this repo for the mistakes a judge would catch: data leakage, metric misuse, features computed differently in training and inference, unpinned dependencies, anything that would not reproduce."

20.3 How to work with it (the part most teams get wrong)

- **Give it the ground truth of your repo, not a description of it.** Paste real rows, real error messages, real confusion matrices. Vague inputs produce vague code.
- **Ask for the baseline before the sophisticated thing.** You need a number to beat by hour 6.
- **Make it argue against itself.** "What is the strongest objection a reliability engineer would raise about this approach?" catches design flaws while they are still cheap.
- **One person owns Claude Code per branch.** Two people running agents on the same working tree at once is how you lose an hour to merge conflicts.
- **Verify every number it reports.** Re-run the script yourself before it goes on a slide. This is non-negotiable for a competition.

- **Use it to learn, not just to type.** "Explain why MAD times 1.4826 equals sigma for a normal distribution" takes 30 seconds and means you can answer the judge who asks.

21. The Pitch

Opening. "Every component in a satellite passes every test on its datasheet. That is the minimum bar, and it is the wrong bar. A part that sits at 45 microamps in a lot where every sibling sits at 10 is not a passing part — it is a failure that has not happened yet. Today it ships, because 45 is less than the datasheet's 50."

Solution. "SENTINEL is an explainable screening layer for burn-in. It learns each production lot's own normal behaviour and flags components whose parametric drift is anomalous relative to their peers. Then it forecasts where each part will be at 168 hours using only its first 24 hours of data — so marginal parts are rejected six days early, freeing burn-in capacity. Every verdict comes with numbered reason codes and evidence a QA inspector can sign."

Proof. "On our validation set, static datasheet limits catch zero percent of latent defects. SENTINEL catches [your number] at a [your number] percent over-rejection budget, and predicts the 168-hour value from hour 24 with a mean absolute error of [your number]."

Close. "Static limits ask whether a part is legal. SENTINEL asks whether it is behaving like its siblings, and where it will be in a week. In a system you cannot repair, that is the difference between a screen and a gamble."

22. Learning Path — what each person should read

Topic	What to look up
Screening standards	MIL-STD-883 Method 1015 (burn-in) and Method 5004 (screening flow); MIL-PRF-38535 Class B vs Class S requirements; ESCC and NASA EEE-INST-002 for the space-parts context
Outlier screening in industry	AEC-Q001 "Guidelines for Part Average Testing"; Static vs Dynamic PAT; the "Good Die in a Bad Neighbourhood" nearest-neighbour method
Iddq testing	Why quiescent current is the most sensitive CMOS defect indicator; delta-Iddq and current-signature methods
Robust statistics	Median absolute deviation, the 1.4826 consistency constant, masking and swamping, Minimum Covariance Determinant
Anomaly detection	scikit-learn's outlier-detection guide: IsolationForest, LocalOutlierFactor, EllipticEnvelope, OneClassSVM
Imbalanced evaluation	Precision-recall curves vs ROC on rare classes; F_g ; cost-sensitive threshold selection
Reliability physics	Arrhenius acceleration factors and activation energies; bathtub curve and infant mortality; electromigration, NBTI, time-dependent dielectric breakdown
Explainability	SHAP TreeExplainer and waterfall plots; the difference between global and local explanation; why rule-based reason codes beat SHAP for an operator audience

23. Day-Zero Checklist

- Repo created; `requirements.txt` pinned; everyone can run `python generate_burnin_dataset.py` and get identical output (fixed seed).
- `CLAUDE.md` written and committed.
- Column naming convention agreed and written down.
- `evaluate.py` exists and is imported by everyone — one scorer, one truth.
- Every team member can define: latent defect, escape, overkill, robust sigma, PDA, safety slope.
- The demo story is written down before any model is trained.
- Someone owns the clock and calls the hour-28 feature freeze out loud.

SENTINEL — team blueprint. Baseline figures in §12 are measured on the synthetic dataset shipped with this document and are reproducible with a fixed seed. All datasets described here are simulated and must be labelled as such in any submission.